

ApiGenie — Release Notes

v5.2.0 — Windows Event Forwarding as a first-class push source

Released June 2026. One body of work in this release: a **complete outbound WEF / WEC push surface** that turns ApiGenie into a fleet of fake Windows Domain Controllers, each pushing real SOAP / WS-Eventing envelopes at a real Windows Event Collector with real auth (Basic over TLS, or mTLS with a Fernet-encrypted client cert bundle on disk). Six phases, one architecture, one new admin tab.

The previous releases let operators *pull* from 16 HTTP APIs and *push* to 16 Log Push destinations. v5.2 closes the matrix on the Windows side: the typical "DC pushes Security 4624 / 4625 / 4768 / Sysmon / PowerShell to the SIEM via a WEF subscription" demo path is now native, profile-aware, RBAC-gated, and visible in a Recent Activity feed alongside the alert-push feed.

At a glance

- **A new RBAC category — WEF Bindings**. Five permissions (`view` / `create` / `modify` / `delete` / `manage`) gate every `/admin/api/wef/*` endpoint. Default entitlements grant `view` to everyone authenticated, `create` / `modify` / `delete` to operators, `manage` to admins.
- **Per-binding storage with encryption at rest**. Each binding row carries the target WEC `host` + `port` + `path`, the auth method (`basic` or `client_cert`), and a `status` block (`last_push_at` / `last_status_code` / `last_error` / `sent_total`). Basic-auth passwords are Fernet-encrypted using the v5.1 key chain (`APIGENIE_SECRET_KEY` or `data/secret.key`). mTLS client-cert PEM bundles ride a parallel per-source storage path (`data/source_certs/wef/<binding-id>.pem.enc`) decrypted to a short-lived temp file only for the duration of an OpenSSL TLS handshake.
- **Async push runner integrated with the FastAPI lifespan**. A module-level singleton (`wef_runner.WEFRunner`) starts on application startup and stops on shutdown. The supervisor task reconciles the enabled set every 5 s; per-binding push tasks pace themselves by `rate_per_min` + `jitter_pct`. Error isolation is per-binding: a broken WEC can never starve another binding or take down the supervisor.
- **Catalog-aware event generation across six channels**. The built-in `EVENT_CATALOG` covers ~200 of the most security-relevant Event IDs across Security / System / Directory Service / DNS Server / Windows-PowerShell-Operational / Microsoft-Windows-Sysmon Operational. Every catalog entry declares `data_fields` so the generator can fill `TargetUserName` / `WorkstationName` / `IpAddress` / `Image` etc. with deterministic values the SIEM-side detection rules correlate on.
- **Profile-driven substitution**. When a binding references a log profile (optional, set in the WEF modal's new Log profile dropdown), the catalog data fields draw from the profile's user / machine pools instead of synthetic placeholders. The same user that shows up in your Okta / CloudTrail /

m365 pulls now shows up in WEF 4624 / 4625 events — cross-source correlation works out of the box.

- **Per-binding & cross-binding push history.** A new in-memory ring (50 most recent attempts per binding + a `_global deque`) feeds a Recent activity card on the WEF tab, identical to the alert-push activity feed. Every `push_once` outcome (success, non-2xx, push exception, factory error) lands in both rings; a `clear_history` endpoint and a per-test fixture keep state hygiene tight.
- **Admin UI tab + Bindings editor modal.** New `WEF Bindings` tab with a binding list, status badges, Start/Stop/Test/Edit/Delete actions, and the Recent activity card. The editor modal hosts `target_host` / `target_port` / `target_path`, an auth-method radio (Basic / mTLS) that swaps in the relevant sub-form, the Channels multi-select, the new Log profile dropdown, and the rate/batch/jitter row. Cert upload (mTLS) is a per-binding POST `/cert` with PEM base64 in the body — never touches the primary binding JSON.

What's new in detail

Phase A — Storage (a81cd22)

A new `wef_bindings.py` module modelled after `webhooks.py` / `alert_push.py`:

- `create_binding`, `get_binding`, `list_bindings`, `update_binding`, `delete_binding`, `set_enabled`, `record_push_result`.
- Owner-scoped visibility (`private` / `public`) using the existing `_can_see_obj` / `_can_write_obj` helpers.
- A status block populated by the runner so the UI sees the latest throughput / error without a second store.
- A new `Category.WEF_BINDINGS` × 5 permissions added to the `accounts` RBAC tables, plus the default entitlement grants.

JSON on-disk at `data/wef_bindings.json`. Basic-auth passwords stored as `basic_password_enc` (Fernet ciphertext) so a leaked snapshot yields nothing usable.

Phase B — Push runner + FastAPI lifespan (57493ca)

`wef_runner.py` adds the bridge between storage and emitter:

- Sync primitives — `push_once(bid)`, `reconcile_sync()`, `stop_all()` — are the deterministic surface tests exercise.
- Async lifecycle — `start()` / `stop()` — wraps the sync primitives in an asyncio supervisor + per-binding push tasks paced by `rate_per_min` + `jitter_pct`.
- Every exception is caught at the per-binding boundary, recorded via `wef_bindings.record_push_result(error=str(exc))`, and the loop continues. The supervisor never dies; one broken WEC can't starve other bindings.

The FastAPI lifespan handler in `app.py` calls `await get_runner().start()` on startup and `await get_runner().stop()` on shutdown, idempotent both ways.

Phase C — Admin REST API + RBAC (1265d5f)

Eight endpoints under `/admin/api/wef/bindings` :

Verb	Path	Permission
GET	<code>/admin/api/wef/bindings</code>	session only
POST	<code>/admin/api/wef/bindings</code>	<code>WEF_BINDINGS.create</code>
GET	<code>/admin/api/wef/bindings/{bid}</code>	session + <code>_can_see_obj</code>
PUT	<code>/admin/api/wef/bindings/{bid}</code>	<code>WEF_BINDINGS.modify</code>
DELETE	<code>/admin/api/wef/bindings/{bid}</code>	<code>WEF_BINDINGS.delete</code>
POST	<code>/admin/api/wef/bindings/{bid}/cert</code>	<code>WEF_BINDINGS.manage</code>
PUT	<code>/admin/api/wef/bindings/{bid}/enabled</code>	<code>WEF_BINDINGS.manage</code>
POST	<code>/admin/api/wef/bindings/{bid}/test</code>	<code>WEF_BINDINGS.manage</code>

Permission requirements live in `_perm_requirement(path, method)` alongside the Webhooks gates. The `/cert` endpoint accepts a JSON body with `pem` base64-encoded; the bundle is validated, encrypted, and persisted via the source's `save_cert_bundle` so the binding JSON never carries any PEM bytes.

Phase D — Admin UI tab (a2e5632)

A new `WEF Bindings` tab in the admin shell with:

- A binding list card. Per row: name, target `host:port` , channel count, current rate, auth-method badge (Basic / mTLS), enabled badge, latest status pill ("never pushed" / "200 · 1234 sent · 12s ago" / "503 · last error: ..."), and four action buttons (Test / Start / Edit / Delete).
- An editor modal with: name, host/port/path, TLS verify checkbox, auth-method radio that swaps Basic ↔ mTLS sub-forms, password field (Fernet-encrypted at rest, never re-sent on GET), cert file picker with a "✓ Certificate already uploaded" indicator on re-edit, Channels multi-select, and a rate / batch / jitter row.
- `tests/test_admin_js_syntax.py` regression — parses the embedded JS via Esprima at test time so a malformed handler can't ship unnoticed.

Phase E — Profile-driven data substitution (8566678)

Closes the explicit `# Real profile-driven substitution ...` out of scope for v5.2 TODO in `sources/windows_event_forwarding.py` . Four layers, one direction:

- A new `_FIELD_RECIPES` table maps each catalog `data_field` (`TargetUserName` , `SubjectUserName` , `AccountName` , `MemberName` , `PrincipalUserName` , `OldTargetUserName` , `NewTargetUserName` , `TargetDomainName` , `SubjectDomainName` , `WorkstationName` , `Workstation` , `ClientName` , `IpAddress` , `ClientAddress` , `TargetServerName`) to (`picker` , `attr`) where `picker` ∈ { `user` , `machine` } selects the `ProfileContext` pool and `attr` is the entity dict key.
- `_materialize_event(entry, record_id, rng, ctx=None)` grows an optional `ctx` kwarg. When supplied and a recipe matches, `ctx.pick_user()` / `pick_machine()` returns an entity or `None`

(noise) and we read the recipe attribute. Missing recipe, noise pick, missing attribute, unset `profile_id` all fall back to the pre-F placeholder. Back-compat: omitting `ctx` produces identical bytes to the old materializer.

- `generate_events(count, ..., ctx=None)` threads `ctx` through to the materializer.
- `WEFEmitter.push_batch` resolves the binding's `config.profile_id` into a `ProfileContext` at emit time (not `__init__`) so a binding survives an out-of-band profile delete without entering an error state. A new `profiles.context_for_profile_id(profile_id, source='wef', ratio=100)` helper short-circuits the source→profile binding table walk that `get_context` does, because WEF stores its profile reference inline on the binding row.
- Admin UI: a new Log profile (optional) form-row in the WEF modal with a dropdown defaulting to "— None — placeholder values". A saved `profile_id` that no longer matches any cached profile (deleted out of band) surfaces as `<id>` (missing – falls back to placeholder) so the operator knows the binding is now in placeholder fallback mode rather than silently switching back to None.

Phase F — Push history ring buffer + activity feed (e1bad2a)

A bounded in-memory ring per binding + a synthetic `_global` deque for the cross-binding feed, modelled 1:1 on `alert_push._history` so the two egress surfaces (alerts / WEF) feel like one product:

- `wef_runner.record_push(binding_id, *, ok, sent, status_code, error, binding_name)` writes to both per-binding and `_global` rings (`deque(maxlen=50)`).
- `wef_runner.get_history(binding_id='_global', *, limit=50)` returns newest-first.
- `wef_runner.clear_history(binding_id=None)` wipes everything (test fixture) or just one binding's ring while pruning matching entries from `_global`.
- `push_once` writes to `record_push` alongside the existing `wef_bindings.record_push_result` write, at every recorded outcome (success, non-2xx, push exception, factory `BindingConfigError`, generic factory exception). The "binding not found" early-return stays history-free.
- Two new GETs: `/admin/api/wef/history` (cross-binding feed, session-only auth like `/admin/api/alerts/history`) and `/admin/api/wef/bindings/{bid}/history` (per-binding, adds the standard `_can_see_obj` ownership gate). Both clamp the `limit` query parameter to `_HISTORY_MAX` so a hostile client can't ask for 10 000 rows.
- New Recent activity card under the WEF Bindings list with a Refresh button. `loadWefBindings` now triggers `loadWefHistory` so a tab open / refresh / Test-push click refreshes both surfaces in one round-trip.

Test plan

- **+78 new regression tests across the six phases.** Total WEF suite: **146 passing in ~3 s** (storage / runner / cert_storage / auth / push_loop / envelope / catalog / event_mix / profile_integration / history / admin_api / admin_js_syntax).
- **Zero regression in the v5.1 surface.** The 31 pre-existing failures in `testotel*` and `testrbac_phase3_avatars` (missing optional `protobuf` / `Pillow` deps) persist as before — they pre-date this branch and are unrelated to WEF.

Upgrade notes

- **No DB migration.** WEF bindings live in their own JSON file (`data/wef_bindings.json`), created lazily on first write. The RBAC tables auto-back-fill the new `Category.WEF_BINDINGS` row on first `accounts._ensure_schema()` run.
- **No new env vars.** The Fernet key chain (`APIGENIE_SECRET_KEY` / `data/secret.key`) introduced in v5.1 protects both Basic-auth passwords and the per-binding client cert bundles. No additional configuration needed.
- **No new volume mount.** Binding rows piggyback on the existing `./data/` mount; per-binding cert bundles land under `./data/source_certs/wef/` , auto-created on first upload.
- **First-time entitlement check.** After upgrading, the admin shell exposes a new `WEF Bindings` tab — visible to every user with at least `view` on the new category. Operators who should also be able to create / start / stop bindings need the `WEF Bindings: manage` permission added to their entitlement. Built-in admins inherit it automatically.

What's next

- **v5.3 — Multi-factor authentication (TOTP).** Bumped from v5.2 to v5.3 to make room for the WEF body of work. Same plan, same rollout phases (mandatory for admin from day one, optional → mandatory for users across the two patch releases). Reuses the v5.1 Phase B Fernet key chain to encrypt MFA seeds. Full plan in [docs/ROADMAP_2026-06-12.md](#) .
- **Unified egress activity feed.** Phase F's history entry shape matches `alert_push._history` 's entry shape on purpose. A follow-up will merge the two feeds into a single "Recent egress activity" pane on the home dashboard, tagged by source label (`wef` / `alert_push` / `webhook`) so the operator can see every outbound from a single page.

Also in v5.2.0 — scenario & Request Inspector consolidation

- **Historical attack-scenario mode removed.** The `historical` mode (v5.1 Phase C) pre-staged events with backdated timestamps in an on-disk backlog that collectors drained on their next poll. The raw telemetry *was* backdated correctly in the lake — but **SentinelOne AI-SIEM detections fire at ingest time, not at the event's backdated time**, so alerts always clustered at "now" regardless of the historical timeline. **Realtime is now the only mode.** Dropped `mode` / `events_per_phase` from scenario create / update / load, removed `pre_stage_historical_events` , `drain_historical_backlog` , the on-disk backlog (`*_backlog.jsonl` / `.idx.json`), `_SOURCE_TS_FIELD` , `_format_ts` , and the historical branch in `start_scenario` ; removed the backlog drain from `detection_rules.inject_detection_events` ; removed the Mode selector, Events/phase input, mode hint, mode pill, and `onScenarioModeChange` from the admin UI. Pre-existing scenarios that still carry the keys on disk keep them (ignored) and run realtime.
- **ransomware_lateral hidden from the template picker.** A `hidden` flag on scenario-library templates keeps it out of the selectable list (SentinelOne's simulated lateral-movement telemetry does not reach the data lake reliably). The template stays resolvable by key so existing scenarios keep working.
- **Request Inspector — "Only my identifiers" filter.** Every traced call is now attributed to the user whose registered source identifier its credential matched (`trace.resolve_caller_id`). A toggle on the Inspector plus a `mine=1` flag on `/admin/api/requests/{source}` and `/admin/api/requests/by-attack/{id}` keep only the caller's own calls, and a new **Caller** column shows who each call resolved to (honours the admin act-as switcher).

New attack scenario — `insider_threat` (EXPERIMENTAL)

- **Disgruntled-employee kill chain, 6 phases across 4 sources.** Mass SharePoint download (M365) → mail auto-forward via transport rule (M365) → insider-threat upload flagged by Netskope → login from an untrusted endpoint (Cisco Duo) → mailbox audit-logging bypass (M365) → Okta high-severity threat. Every phase's `field_overrides` carry the exact discriminators of a **real shipped SentinelOne platform rule** discovered on the validation tenant, and `target_rules` document each rule's `s1ql`, `id`, and shipped status.
- **Shipped selectable but flagged experimental.** A new `experimental` flag on scenario-library templates (sibling to `hidden`) keeps the template fully usable while the Create-Scenario modal renders an **EXPERIMENTAL** badge on both the dropdown option and the description. Rationale: the phase engineering is complete and validated field-by-field, but **end-to-end alert firing is still being hardened on two sources** — the Netskope "Malware Upload" rule keys on `unmapped.activity`, which the customer's collector does not land, and the earliest M365 phases can fall between collector polls on short runs. BEC and Cloud Account Takeover remain the fully-validated reference scenarios.
- **Reusable S1 grounding.** S1QL string `=` is **case-sensitive**; `unmapped.*` fields are raw pass-through (emit the exact value+case to match), while **mapped** fields (e.g. Cisco Duo `result→status`, `reason→status_detail`) go through known-value lookups that uppercase/drop out-of-vocabulary values. Phase values were chosen accordingly.

Attack scenarios require their shipped rules **ENABLED** on the tenant

- **Alerts only fire if the target platform rule is enabled.** Several BEC / Cloud Account Takeover / `insider_threat` target rules ship **Disabled** by default on a SentinelOne tenant (e.g. *Proofpoint Impostor Email Unblocked*, *Okta Impersonation Session Initiated*, *Azure User Added to a Highly Privileged Built-in Role*, *Office 365 Service Principal Addition*). If they are Disabled, the matching telemetry lands in the lake but produces **no alert** — this is a tenant configuration state, not an ApiGenie regression.
- **Runbook scripts.** `scripts/toggle_insider_rules.py` and `scripts/toggle_bec_ct_rules.py` enable (default), `--disable` (rollback), or `--status` (read-only) the relevant shipped rule ids using the saved console settings. See [docs/ATTACK_SCENARIOS.md](#) for the full per-scenario rule list and enablement steps.

v5.1 — Security hardening + time-shifted attack stories

Released June 2026. Three bodies of work land in one release, all under a consistent theme: **shrink the platform's secrets footprint and give operators control over when an attack happened**, without changing how collectors are wired.

1. **Phase A — Per-user S1 console moves to the browser.** Each user's SentinelOne console URL and API token now live exclusively in `localStorage`, sent on every authenticated request as `X-S1-Console-URL` / `X-S1-Console-Token` headers, resolved server-side via a per-request `ContextVar`. The server never writes a per-user S1 token to disk again.

2. **Phase B — Fernet for the admin-global S1 token.** The single admin-global S1 API token (used when no per-user override is sent) is now Fernet-encrypted at rest, with a key chain loaded from `APIGENIE_SECRET_KEY` or `data/secret.key`. The same key chain will protect MFA seeds in v5.2.
3. **Phase C — Historical attack scenarios + auto-generated setup notes.** Scenarios gain a `mode` switch: *realtime* keeps the existing forward-running scheduler, *historical* pre-stages every event with backdated timestamps so the full attack story is immediately visible to collectors. Every scenario now also carries an expandable, auto-generated **Setup notes** card that tells the operator exactly which collectors / push profiles to configure for the run to play out end-to-end.

At a glance

- **Per-user S1 token never touches the server filesystem again.** Card on the *My account* page reads/writes `apigenie.s1.console_url` and `apigenie.s1.api_token` in the browser; a global `fetch` wrapper installed at admin shell load injects the two headers on every authenticated XHR. The middleware in `app.py` reads them into a `ContextVar` for the duration of the request; `s1_detection_library` resolves them transparently in `_resolved_settings()`.
- **Server-side S1 columns gone.** `users.s1_console_url` and `users.s1_api_token` are no longer read or written; the old `/admin/api/me/s1-console` GET / PUT / DELETE endpoints return `404` / `405`. Existing data on disk is harmless and ignored.
- **Admin-global S1 token Fernet-encrypted.** `crypto.py` wraps `cryptography.fernet`. Key precedence: `APIGENIE_SECRET_KEY` env var (preferred for production), else `data/secret.key` (auto-generated on first launch, persisted, mode `0o600`). The admin-global S1 token ciphertext lives in `data/s1_settings.json` under `api_token_enc`.
- **Historical scenario mode.** New `mode: "realtime" | "historical"` field on every scenario. Realtime is the default and keeps today's behaviour. Historical, on launch, computes every event for the full duration once and stamps each with a `_ts` drawn uniformly from `[now - duration, now]`. The events land in a per-scenario backlog file under `data/scenarios/<scenario-id>_backlog.jsonl`, with a sidecar `<scenario-id>_backlog.idx.json` carrying per- (source, caller) drain cursors so each user's collector replays the story independently.
- **inject_detection_events drains the backlog.** When a collector polls a source, the detection-rules layer transparently prepends any pre-staged events for the resolved caller before returning the live batch. Existing realtime scenarios are unaffected.
- **Visibility on scenarios.** New `visibility: "private" | "public"` field. Private scenarios only deliver their events to the launching user's caller token; public scenarios deliver to every caller. Both modes honour visibility — useful for parallel demos on the same ApiGenie instance.
- **Auto-generated Setup notes.** On create / update, every scenario computes a `setup_notes` block that lists every distinct source the scenario touches and, for each source, surfaces its kind (pull / push / bus / unknown), endpoint, auth mechanism, and configuration hints. Rendered as an expandable card on each scenario row in the admin UI; intentionally excluded from `export_scenario()` and regenerated by `import_scenario()` so the notes always reflect the ApiGenie that's hosting the run.
- **Scenario card pills.** Each scenario row now shows two status pills next to the duration / phase count: green `REALTIME` vs purple `HISTORICAL`, and amber `PRIVATE` vs muted `PUBLIC`.

- **+45 regression tests.** Total: **640 passing** (up from v5.0's 595). New files: `tests/test_scenarios_historical.py` (schema defaults, pre-staging, per-source and per-caller drain, visibility enforcement, lifecycle, `inject_detection_events` integration), `tests/test_scenarios_setup_notes.py` (creation, dedup, known / unknown source hints, update regeneration, export / import integration). `tests/test_rbac_phase35_self_service.py` and `tests/test_rbac_phase35_endpoints.py` were rewritten end-to-end to cover the new header-based S1 override and to assert the removed endpoints stay removed.

What's new in detail

Phase A — Per-user S1 console moves to the browser

The per-user SentinelOne console URL + API token were previously persisted in two SQLite columns on `users`, encrypted only by SQLite file ACLs. v5.1 removes them entirely. The new model:

Layer	Where the value lives	Lifetime
Input	<code>localStorage</code> keys <code>apigenie.s1.console_url</code> , <code>apigenie.s1.api_token</code>	Until the user clears them
Wire	Headers <code>X-S1-Console-URL</code> , <code>X-S1-Console-Token</code>	Single request
Server	<code>ContextVar</code> set by middleware, read by <code>s1_detection_library._resolved_settings()</code>	Single request
Disk	Nothing	—

The admin shell installs a one-time global `fetch` wrapper on load so every authenticated XHR carries the two headers automatically — no caller-side opt-in required. The *My account* page replaces the old form with a browser-only card that reads, saves, and clears `localStorage` and explains the storage model inline. Clearing the browser data clears the override; signing out does not (the user can keep the credentials across sessions).

`_build_asset_resolver_for_session` no longer reads per-user columns; it asks `s1_detection_library._resolved_settings()` for the effective URL + token and returns `None` if neither a global token nor a request-scoped override is available.

Phase B — Fernet for the admin-global S1 token

The single admin-global S1 API token (the fallback used when no per-user override is sent) was previously written plaintext to `data/s1_settings.json`. v5.1 introduces `crypto.py`:

```
def encrypt(plaintext: str) -> str: ... # returns base64 Fernet token
def decrypt(ciphertext: str) -> str: ... # raises if key missing / tampered
```

Key resolution at module import time:

1. `APIGENIE_SECRET_KEY` env var (32-byte URL-safe base64) — preferred for any production / shared environment.

2. `data/secret.key` — auto-created on first launch if absent (`Fernet.generate_key()` , mode `0o600`). Suitable for single-host demos.

The admin-global S1 token now persists as `api_token_enc` in `data/s1_settings.json` . Legacy `api_token` plaintext is read once on load, re-encrypted, and removed from disk on the next write. The same key chain is what v5.2 will use to encrypt MFA seeds — keeping the key-management surface to one file / one env var.

Phase C — Historical attack scenarios

Until v5.0, an attack scenario was a forward-running clock: launch at `t=0` , the scheduler walked the phases over wall-clock, and events appeared only when collectors polled *during* an active phase window. That works for live demos but is awkward for investigations and post-incident replays — if nobody polled for an hour, that hour was empty.

v5.1 adds a second mode. On the create / edit modal, a new **Mode** dropdown sits next to the existing Name / Duration row:

- **Realtime** (*default, unchanged*) — phases activate over wall-clock, events fire as collectors poll inside each phase window.
- **Historical** — on launch, ApiGenie iterates every phase, computes the expected event count for that phase (`events_per_phase` if set, else `phase_duration_seconds * periodicity / 60 * APIGENIE_SCN_ASSUMED_RATE_PER_MIN` , default rate `12`), draws each event's timestamp uniformly from the phase's window inside `[now - duration, now]` , and writes every event to `data/scenarios/<scenario-id>_backlog.jsonl` . A sidecar `<scenario-id>_backlog.idx.json` carries a `{ source → { caller_id → offset } }` cursor map so that each (source, caller) pair drains the backlog independently — caller A consuming Okta events does not skip ahead for caller B, and consuming Okta does not affect Proofpoint.

The drain is wired into `detection_rules.inject_detection_events` : on every pull, the function asks `attack_scenarios.drain_historical_backlog(source)` for events visible to the resolved caller, prepends them to the live batch, and advances the cursor. Realtime scenarios skip the drain entirely (the backlog file does not exist).

`start_scenario()` branches on `mode` . Historical scenarios skip scheduler activation and call `pre_stage_historical_events()` instead; their status reports show `mode=historical` and `events_pre_staged=N` in the API and in the UI card. `delete_scenario()` removes both the `.jsonl` and the `.idx.json` sidecar so a re-launch of the same scenario starts from a clean backlog.

Setup notes — every scenario tells you how to configure it

`create_scenario()` and `update_scenario()` now compute a `setup_notes` block from the phases' sources. For every distinct source the scenario touches, the block records: `source` , `kind` (`pull` | `push` | `bus` | `unknown`), `endpoint` (when known), `auth` hint, recommended `options` , and free-form `notes` . A built-in hint table covers every catalog-aware source ApiGenie ships; unknown sources fall through to a placeholder reminding the operator to wire them by hand. The block is rendered as an expandable card on each scenario row in the admin UI — collapsed by default, with the per-kind chip colour-coded so an operator can scan a multi-source scenario at a glance and tell which collectors are pull-pulled, push-fed, or bus-published.

`setup_notes` are **excluded from** `export_scenario()` so a scenario exported from instance A and re-imported on instance B regenerates the notes against B's source catalog — the notes always reflect the host that's running the scenario, never the host that authored it.

RBAC

No new categories, no new permissions. Scenarios continue to use the `attack_scenarios:*` permission set introduced in v5.0. The new `visibility=private` field adds a runtime owner-token scoping at drain time (the backlog drain matches the caller against `owner_id`) and does not require a separate capability.

Migration

- **No DB migration.** Scenarios are JSON-on-disk; `_load_scenarios()` back-fills `mode=realtime`, `visibility=public`, `owner_id=null`, `events_per_phase=null` for any pre-v5.1 file. The `users.s1_console_url` / `users.s1_api_token` columns are simply ignored by the code path; the columns themselves can stay or be dropped at the operator's leisure.
- **No new mandatory env var.** `APIGENIE_SECRET_KEY` is *preferred* but optional; absent it, `data/secret.key` is auto-generated on first launch. `APIGENIE_SCN_ASSUMED_RATE_PER_MIN` is optional and only affects historical mode's auto-derived event counts.
- **No new volume mount.** Backlog files live under the existing `./data/scenarios/` mount.
- **Pre-v5.1 scenarios have no `setup_notes`** until they are opened in the editor and re-saved (or re-imported). The card will show a short "No setup notes available" placeholder in the meantime — cosmetic only, behaviour is unchanged.
- **Per-user S1 tokens previously persisted server-side are dropped** on first read. Affected users will see an empty S1 console card on *My account* on next login and need to paste their console URL + token once — they will then live in `localStorage` for that browser.

Upgrade procedure

```
git pull
docker compose build apigenie
docker compose up -d apigenie
# Optional sanity:
docker exec apigenie python -m pytest -q \
    tests/test_scenarios_historical.py \
    tests/test_scenarios_setup_notes.py \
    tests/test_rbac_phase35_endpoints.py \
    tests/test_rbac_phase35_self_service.py
```

No data wipe, no re-bootstrap, no admin re-login needed. If `APIGENIE_SECRET_KEY` is set in the environment, `data/secret.key` is ignored.

What's next

- **v5.2 — Multi-factor authentication (TOTP).** Single feature. Mandatory for admin from day one, optional → mandatory for users across v5.2.0 → v5.3.0. Reuses the v5.1 Phase B Fernet key chain

to encrypt MFA seeds. Full plan in [docs/ROADMAP_2026-06-12.md](#).

- **Windows Event Forwarding push source.** Outbound WS-Management / WS-Eventing emitter that mimics a Domain Controller pushing ~200 of the most security-relevant Event IDs (Security + System + Directory Service + DNS-Server + Windows-PowerShell-Operational + Sysmon) to an external WEC, with per-binding mutual-TLS or TLS+Basic auth and per-source PEM upload encrypted with the Phase B Fernet key. Full event-mix participation.

v5.0 — *Reshape every source, fire from any pane*

Released June 2026. Four bodies of work land in one release, all under a consistent theme: **giving operators control over what the platform emits and where it lands**, without editing source code.

1. **Webhooks** — a templated outbound HTTP request composer that lets any signed-in user fire shaped JSON payloads at third-party SIEM / SOAR / arbitrary HTTPS endpoints, with profile-aware substitution (users / machines / C2 / malware / mail senders) and `{{custom.<key>}}` send-time variables.
2. **Attack Scenarios — Phase 2 + 3.** Multi-source MITRE-mapped campaigns gain a custom builder + import/export, a per-scenario event log, a cross-source `attack.id` search with reveal nav, and an exportable timeline.
3. **Event Mix** — a new admin surface (REST + UI disclosure on every bindings card) that re-weights or disables individual event types per source, with per-user override scoping and a `source_bindings` -piggybacked RBAC model.
4. **Event Mix per-source rollout — 21 / 21 sources.** Every catalog-aware source ships with an `EVENT_CATALOG` and threads `event_mix.apply()` through its generator. Includes a source-id alias layer (`entra_id` → `azure_ad`, `defender` → `microsoft_defender`) and the relocation of `azure_platform` (Event Hubs / Kafka) from `publishers/kafka_publisher.py` into a proper `sources/azure_platform.py` module so the Kafka producer participates in the mix surface too.

At a glance

- **Webhooks composer.** New `Webhooks` RBAC category × 5 permissions. Templated body editor with profile substitution (`{{profile.user.username}}`, `{{profile.machine.hostname}}`, `{{profile.c2.fqdn}}`, ...) and free-form `{{custom.<key>}}` variables resolved at send-time. Saved profiles, send history, retry, and a "Send & inspect" debug mode with full request + response capture. Full reference: [docs/WEBHOOKS.md](#).
- **Custom Attack Scenarios.** Build scenarios in the admin UI (add phases, pick sources, set fan-out and timing), export them as JSON, re-import them on another instance. The shipped library doubles as starter templates.
- **Per-scenario event log + cross-source search + timeline export.** A dedicated event-log panel scoped to each scenario run; an `attack.id` search anywhere in the admin / portal that jumps to the originating scenario / phase with deep-links into the source's hit pane; a chronological timeline

export (JSON) that joins every emitted event with its phase metadata for post-mortem decks or SIEM hunts.

- **Event Mix admin surface.** New REST endpoints (`/admin/api/event-mix/sources` , `/admin/api/sources/{src}/event-catalog` , `/admin/api/source-event-mix/{src}` PUT/DELETE) plus an inline disclosure card on every mix-aware source on the bindings page. Sliders + per-id toggles + Save / Reset. RBAC piggy-backs `source_bindings:modify` so users who can shape what a source emits also get the mix.
- **Per-user override scoping.** Admin without acting-as → writes the global mix. Real user (or admin acting-as) → writes a private override that only shadows the mix for that user. The same pattern already used by source profiles, identifiers, and bindings.
- **21 / 21 sources wired.** Every source listed in the v4.1 Phase 5 catalog ships with an `EVENT_CATALOG` and threads through `event_mix.apply()` . Last batch: `cloudflare` , `snyk` , `tenable` , `wiz` , `zscaler_zpa` , plus the relocation of `azure_platform` .
- **Source-id alias layer.** The bindings UI uses Microsoft's marketing names for two sources (`entra_id` , `defender`) but the Python modules live under `azure_ad` and `microsoft_defender` . `sources/__init__.py` exposes `SOURCE_ID_ALIASES` + `canonical_source_id()` ; every event-mix endpoint canonicalises before touching storage so a save against the Entra ID card persists under the canonical key. The catalog endpoint dual-emits each aliased catalog so the UI lookup hits either way.
- **azure_platform relocated.** The Kafka producer for Event Hubs previously inlined its templates + generator in `publishers/kafka_publisher.py` . Moving it into `sources/azure_platform.py` (with a 14-entry `EVENT_CATALOG` spanning Azure Monitor diagnostic settings + Entra ID activity logs) lets it participate in the mix surface without touching the Kafka batch loop.
- **~30 new regression tests across the four bodies of work.** Webhooks endpoints + substitution + RBAC; scenario builder validation + import/export round-trip; per-scenario event log + timeline export shape; the full Event Mix story (catalog presence + default-weight sums for all 21 sources, alias resolution, alignment between `EVENT_CATALOG` and internal templates, empirical disable proofs that actually generate events and assert the disabled markers are gone). Total regression: **595 passing.**

What's new in detail

Webhooks

A new top-level admin feature with its own RBAC category (`Webhooks` × `view` , `create` , `modify` , `delete` , `send`). The composer renders a profile-aware editor: the bound log profile's entities (`users` , `machines` , `c2_servers` , `malware_samples` , `mail_senders`) are exposed as template variables and substituted into the request URL, headers, and body at send-time. A `{{custom.<key>}}` namespace lets the operator stash one-off values (a JIRA ticket id, a Slack channel name, a specific IP) in a bottom pane and reference them from the same template.

Saved webhook profiles are scoped by `owner_id` + visibility (`private` / `public`) using the same model as Log Push profiles. Send history and request-response inspection ride on the existing `REQUEST_TRACE` plumbing. See [docs/WEBHOOKS.md](#) for the template language, the substitution rules, and the REST surface.

Attack Scenarios — Phase 2 (custom builder + import/export)

Operators can compose new scenarios end-to-end in the admin UI: add phases, pick which sources each phase emits to, set fan-out (events per source per phase), set inter-phase timing (immediate / fixed delay / jittered range), and preview the resulting `attack.id` / `phase.id` stamping before running. The result is a JSON document with the same schema as the shipped library scenarios, so import is a drag-and-drop of an exported file. A round-trip test ensures the export → import path preserves every field.

Attack Scenarios — Phase 3 (event log + cross-source search + timeline)

Phase	What it does	Where it lives
3.1 — Per-scenario event log	A new panel on each scenario detail page that shows every emitted event (with <code>attack.id</code> , <code>phase.id</code> , source, timestamp). Same renderer as the source hit panes, scoped to the scenario run.	<code>attack_scenarios.py</code> , admin UI tab
3.2 — Cross-source <code>attack.id</code> search	A search box anywhere in the admin / portal that takes an <code>attack.id</code> and jumps directly to the originating scenario, with deep-links into every source's hit pane filtered to that id.	<code>attack_scenarios.py</code> resolver + admin UI reveal nav
3.3 — Exportable attack timeline	A chronological JSON timeline that joins every emitted event with its phase metadata, ready to drop into a post-mortem deck or a SIEM hunt.	<code>attack_scenarios.py</code> exporter

Design doc: [docs/ATTACK_SCENARIOS.md](#).

Event Mix — admin surface

The full design lives in [docs/EVENT_MIX.md](#). The storage layer (`event_mix.py`) writes overrides under the same per-user JSON pattern as `source_profiles.py`. The source registry (`sources/__init__.py`) discovers every module that declares an `EVENT_CATALOG` at module scope and exposes them through three lookup helpers used by both the admin API and the test suite.

Component	Where it lives	What it does
Storage	<code>event_mix.py</code> (<code>set_mix</code> , <code>get_mix</code> , <code>reset_mix</code> , <code>list_mixes</code> , <code>list_mixes_for_user</code> , <code>merge_catalog_with_mix</code> , <code>apply</code>)	Per-user JSON overrides; default-weight resolution; "all disabled → fall back to defaults" guard so an overzealous disable never produces an empty response.
Source registry	<code>sources/__init__.py</code> (<code>iter_source_modules</code> , <code>get_event_catalog</code> , <code>get_event_catalogs</code> , <code>SOURCE_ID_ALIASES</code> , <code>canonical_source_id</code>)	Lazy module iteration, alias layer for bindings-UI ids that differ from Python module filenames, dual-emit so the bindings UI lookup always hits.
Admin REST	<code>admin.py</code> (<code>/admin/api/event-mix/sources</code> , <code>/admin/api/sources/{src}/event-catalog</code> , <code>/admin/api/source-event-mix</code> × <code>GET/PUT/DELETE</code>)	Canonicalises every path-param <code>source</code> before touching storage, returns enriched catalog (default + effective weight + <code>enabled</code>), surfaces <code>own</code> flag so UI knows which overrides are editable.
Admin UI	<code>admin.py</code> bindings page	Inline disclosure on every mix-aware source card; slider + toggle per id; Save / Reset; shows "(global, read-only)" tag when the effective mix is inherited from the admin global.

Event Mix — per-source rollout

Source	Mix-axis	Entries
cisco_duo	Authentication outcome x reason	5
okta	User event type	7
proofpoint	Mail event type	4
aws_cloudtrail	Action category	6
aws_guardduty	Finding category	7
aws_waf	Action x terminating rule	7
azure_ad (UI: entra_id)	Two endpoint families — directoryAudits (6) + signIns (5)	11
microsoft_defender (UI: defender)	MITRE-anchored alert	5
m365	Top-level event category	14
mimecast	SIEM API log type	8
cato	GraphQL eventsFeed + auditFeed category	4
darktrace	Two endpoint families — /modelbreaches (6) + /aianalyst/incidentevents (4)	10
gcp_audit	Cloud Audit Logs category	4
netskope	v2 alerts API alert_type	11
sentinelone	Threat classification	9
cloudflare	Logpush dataset family	8
snyk	Issue severity x type	4
tenable	Vuln plugin	4
wiz	Issue type	7
zscaler_zpa	ZPA log stream	5
azure_platform	Event Hubs category x operation	14

Total: 21 / 21 sources, 150 event types under operator control.

Source-id alias layer

The bindings UI uses Microsoft's marketing names for two sources but the Python modules kept their original filenames:

UI binding id	Canonical module	Reason
entra_id	<code>sources/azure_ad.py</code>	Microsoft renamed Azure AD to Entra ID (2023).
defender	<code>sources/microsoft_defender.py</code>	Defender XDR (Endpoint + Identity + Cloud Apps).

Without an alias layer the bindings page's `_mixAwareSources[src]` lookup would never hit for these two cards. `sources/__init__.py` exposes `SOURCE_ID_ALIASES` and `canonical_source_id()`; every event-mix endpoint canonicalises before touching storage so the override key matches the source-side resolver. The catalog endpoint dual-emits each aliased catalog under both ids so the UI lookup succeeds with either label.

azure_platform relocation

The Kafka producer for Event Hubs lived in `publishers/kafka_publisher.py` with an inline 15-template list and an inline `_generate_azure_event` function. v5.0 relocates the templates + generator into `sources/azure_platform.py` (with a 14-entry `EVENT_CATALOG`, a `_AZURE_TEMPLATES` dict keyed by catalog id, and the same `event_mix.apply()` → `weighted_choice` pipeline used by every other catalog-aware source). The publisher imports `generate_azure_event` from there; the batch loop and the Kafka topic creation are untouched.

RBAC

Category	New in v5.0	Permissions used
Webhooks	✓	<code>view</code> , <code>create</code> , <code>modify</code> , <code>delete</code> , <code>send</code>
Source Bindings	↻ extended	<code>source_bindings:modify</code> now also gates Event Mix overrides — "I can shape what this source sends to my collector".
Attack Scenarios	↻ extended	Phase 2 custom builder + phase 3 timeline export use the existing <code>attack_scenarios:create</code> / <code>:modify</code> / <code>:view</code> permissions; no new entitlement keys.

Migration

- **Existing v4.1 deployments:** zero data migration. Event Mix storage is opt-in per user; the first save creates the file. The alias layer is read-only and applied at runtime.
- **azure_platform consumers:** the Kafka topic, the published JSON shape, and the legacy uniform distribution are byte-identical to v4.1. The relocation is internal.
- **Custom sources:** if you maintain a fork with a private source, declare `EVENT_CATALOG` at module scope to opt into the mix surface. No registration call needed — `sources/__init__.py` discovers it via `pkgutil.iter_modules`. See [docs/EVENT_MIX.md](#) §"Adding a mix-aware source".

Acknowledgements

Track B (per-source rollout) batches landed in 6 increments — every commit is a green-CI snapshot of the rollout state. The empirical disable tests in `tests/test_event_mix_sources.py` pick a unique marker per

source (event_type == 'health' for ZPA, category == 'RiskyUsers' for Azure Platform, outcome.result == 'FAILURE' for Okta brute-force markers, ...) and assert that disabling the catalog id actually drops every event carrying that marker — the resolver is wired end-to-end, not just at the storage layer.

v4.1 — *OpenTelemetry, both directions*

Released June 2026. Symmetric OpenTelemetry support lands in apigenie: a new **OTLP push-sink listener** that accepts exports *into* apigenie over OTLP/HTTP (port 443) and OTLP/gRPC (port 4317), and an **OTLP push-egress transport** in the Log Push framework that streams synthetic topics or uploaded replay files *out* to any OTLP collector. Together they let apigenie sit anywhere in a telemetry pipeline — as a collector, as a producer, or both at once (a single instance can even round-trip its own exports for smoke / demos).

This release also fixes a **silent admin/portal nav leak** caused by a Python-vs-JS string-escape bug in the listener hit pane (a runaway `<script>` that killed `gatePortalRole()` and left every `data-portal="admin"` item visible in `/portal`). The companion fix — installing `node` in the production image — turns `tests/test_admin_js_syntax.py` from a silently-skipped no-op into a real CI gate that catches this entire bug class going forward.

At a glance

- **OTLP/HTTP push sink** — `POST /listener/<id>/v1/{logs,metrics,traces}` with `application/x-protobuf` or `application/json`, returning spec-compliant `{"partialSuccess": {}}` 200 acks.
- **OTLP/gRPC push sink** — `grpc://<host>:4317` (TLS-terminated by nginx, plaintext h2c upstream) speaking `opentelemetry.proto.collector.{logs,metrics,trace}.v1.*.Export`. Multi-tenant routing via metadata: `x-apigenie-listener-id`, the Grafana-compatible `x-scope-orgid`, a unique bearer token, or sole-sink fallback.
- **OTLP push-egress transport.** Two new transports on the Log Push framework — `otlp_http` and `otlp_grpc` — that marshal any push source into an `ExportLogsServiceRequest` and ship it to an external collector (or to apigenie's own push-sink listener for a self-round-trip). Five new push sources land alongside: `synthetic_endpoint`, `synthetic_identity`, `synthetic_cloud`, `synthetic_network` (each reusing the listener's synthetic generators byte-for-byte), and `replay_file` (streams an uploaded log file via the existing replay engine with time-shift anchoring).
- **Decoded hit preview.** Every accepted export records a `otlp_preview` block on the hit (resource attributes as KV chips + first N records as a JSON-ish list), rendered inline in the existing  **Hits** pane. Use `decode_preview: false` to opt out for high-volume streams.
- **Wizard tiles, both sides.** The Listeners wizard step 3 adds an **OTLP push sink** radio next to *Synthetic* and *Replay*; the Log Push wizard adds **OTLP/HTTP** and **OTLP/gRPC** transports plus the 5 new sources with smart defaults (auto port 4317 for gRPC, auto `/v1/logs` for HTTP, replay-file picker reuses the existing `/admin/api/replays` endpoint).

- **~70 new regression tests.** `tests/test_otel_listener.py` (47) covers the inbound half — data-model validation, codec wiring, decoder paths (logs/metrics/traces, malformed, oversize, truncation), HTTP dispatch (auth, path mismatch, decode opt-out), and the full gRPC server (routing tiers, bearer auth, signal mismatch, `NOT_FOUND` on ambiguity). `tests/test_otel_pusher.py` (~22) covers the outbound half — registry, profile model, source modules, request builder (severity / timestamp / whitelist mapping), HTTP transport (stub server), gRPC transport (real `listeners_grpc` server on an ephemeral port), and end-to-end dispatcher routing.
- **Two live smoke scripts.** `scripts/smoke_otlp.py` covers inbound: creates a push-sink listener, pushes one OTLP/HTTP and one OTLP/gRPC export, asserts both decoded previews land, cleans up. `scripts/smoke_otlp_egress.py` covers outbound + round-trip: creates a push-sink listener, runs an OTLP/gRPC push profile against it, then an OTLP/HTTP push profile against it, asserts at least 10 decoded events land on the listener hit pane, cleans up. Both run via `docker exec apigenie python /app/scripts/smoke_otlp{,_egress}.py`.

What's new in detail

OpenTelemetry listener kind

The full design lives in `docs/OTEL_LISTENER.md`. The data model adds one new optional field to the `Listener` dataclass (`push_sink: PushSinkSpec | None`) which is mutually exclusive with `synthetic` and `replay` — validated server-side. Two new codecs land at the same time: `otlp_proto` and `otlp_json`.

Component	Where it lives	What it does
Data model + validation	<code>listeners.py</code> (<code>PushSinkSpec</code> , <code>ALLOWED_CODECS_V1</code> , <code>validate_listener_payload</code> , <code>make_hit</code>)	Three-way XOR between data-source kinds, OTLP-specific constraints (gRPC $\Rightarrow$ <code>otlp_proto</code> , <code>path</code> must start <code>/v1/<signal></code>), <code>otlp_preview</code> field on hits.
Decoder	<code>listeners_otlp.py</code>	Lazy-loaded protobuf classes, best-effort decode of the body into a compact preview dict (resources $\times$ records), 4 MB body cap, structured <code>decode_error</code> reporting (truncation, malformed, oversize).
HTTP dispatch	<code>app.py</code> <code>listener_dispatch</code> (push_sink branch)	Routes <code>POST /listener/<id>/v1/<signal></code> to the OTLP path, runs the existing auth / chaos / rate-limit pipeline, acks with the spec-compliant empty <code>Export*ServiceResponse</code> .
gRPC server	<code>listeners_grpc.py</code> (new)	<code>grpc.aio.Server</code> on its own thread + event loop. Three <code>Export</code> servicers (Logs / Metrics / Trace). Routing precedence: explicit listener id $\rightarrow$ Grafana <code>x-scope-orgid</code> $\rightarrow$ bearer-token-unique $\rightarrow$ sole-sink. Mirrors the HTTP path for auth + chaos + hit recording.
FastAPI lifespan	<code>app.py</code> (lifespan hook)	Starts the gRPC server on app startup (gated by <code>APIGENIE_OTLP_GRPC_ENABLED</code> , default <code>true</code>), stops it on shutdown. Failure is logged but non-fatal — apigenie keeps running with the HTTP half operational.
nginx TLS terminator	<code>nginx/nginx.conf.template</code>	New <code>server { listen 4317 ssl; http2 on; grpc_pass grpc://apigenie:4317; } block</code> , modelled on the existing Pub/Sub-emulator 8443 pattern. Uses the same Let's Encrypt cert as 443 / 8443.
Admin UI wizard tile	<code>admin.py</code>	Step-3 radio + signal/protocol/decode-preview/max-records fields. Codec dropdown gains <code>otlp_proto</code> / <code>otlp_json</code> . Hit pane renders a 📄 OTLP pill + collapsible decoded preview with resource attributes and records.
Tests + smoke	<code>tests/test_otlp_listener.py</code> , <code>scripts/smoke_otlp.py</code>	47 unit + integration tests. End-to-end smoke against a live container.

The push-sink ingest path is **not** behind RBAC — the listener's own `auth` spec (none / bearer / basic / `x_api_key`) decides who can push, exactly like synthetic and replay listeners. Listener *configuration* (create / edit / delete / clone) reuses the existing `Category.LISTENERS  $\times$  Permission`. `{CREATE,MODIFY,DELETE,VIEW}` model with `owner_id` + `visibility` scoping. No new admin endpoints were introduced.

RBAC regression fix

A Python `'\n'` (real newline) was emitted inside a JS string literal in `admin.py`'s `_renderOtlpPreview` helper, producing an unterminated string at JS parse time. The whole `<script>` block aborted, taking with it the IIFE that hides `data-portal="admin"` nav items in `/portal`. Visible symptom: Troubleshooting (Intrusions, Investigations, Container Logs) and System (System Settings) sections rendered for non-admin users — despite the server-side HTML being correct and `_portal_role_guard` still rejecting any admin API call from a user session. Fixed by replacing `'\n'` with `'\\n'` (the convention every other JS string in `admin.py` already used).

OpenTelemetry push-egress transport

The outbound mirror of the listener feature — documented in [docs/OTEL_LISTENER.md §15](#). The Log Push framework (`log_pusher.py`) gains two new transports without disturbing the existing JSON / HEC / Syslog hot paths:

Component	Where it lives	What it does
OTLP marshaller	<code>otlp_pusher.py</code> (new)	Dict→LogRecord mapping with severity normalisation (OTel spec bands), timestamp unit autodetect (s / ms / us / ns), whitelisted scalar attribute promotion, JSON body fallback for nested structures, and per-record truncation.
HTTP transport	<code>otlp_pusher._send_http</code>	POSTs <code>application/x-protobuf</code> to <code>/v1/<signal></code> with optional Bearer auth and <code>X-Apigenie-Listener-Id</code> / <code>X-Scope-0rgid</code> routing headers for in-cluster delivery.
gRPC transport	<code>otlp_pusher._send_grpc</code>	Unary <code>LogsService/Export</code> call with the same routing metadata. <code>grpc.insecure_channel</code> for plaintext h2c (the in-cluster default), <code>grpc.secure_channel</code> for external TLS.
Push framework wiring	<code>log_pusher.send_event</code>	Lazy-imports <code>otlp_pusher</code> and dispatches <code>otlp_*</code> transports to it; everything else (JSON/Syslog/CEF) is unchanged. New profile fields: <code>replay_file_id</code> , <code>otlp_signal</code> , <code>otlp_listener_id</code> . Stateful source contract via <code>make_iterator(profile)</code> so the replay source can stream a file across calls without per-event re-parsing.
5 new sources	<code>push_sources/synthetic_*.py</code> , <code>push_sources/replay_file.py</code>	The 4 synthetic topics reuse <code>sources.synthetic.{endpoint,identity,cloud,network}.generate(1)</code> directly so a record emitted from a listener-side synthetic topic is byte-identical to the same topic streamed via the push side. The replay source streams an admin-uploaded file through <code>replay.stream()</code> with time-shift anchoring.
Wizard	<code>admin.py</code> push profile editor	New transport dropdown options, OTLP-only field group (signal + listener id), replay-file picker (shown only when <code>source_type=replay_file</code> , lazy-loads via <code>/admin/api/replays</code>), and smart defaults per transport.

The egress half is **opt-in per push profile** — no new global env vars, no new sockets opened by apigenie itself (it's a client of the destination). Existing push profiles keep working unchanged because the new fields default to safe values (`otlp_signal="logs"` , `replay_file_id=None` , `otlp_listener_id=None`).

CI hardening — no more silent skips

- **nodejs in the Docker image** so `tests/test_admin_js_syntax.py` (a `node --check` lint of every embedded `<script>` block) actually runs in CI. It had been silently skipping for the entire v4.0 lifetime — long enough for the v4.1 JS bug above to slip in undetected. Future bugs of the same shape will fail this test loudly.
- **pytest + pytest-asyncio in the Docker image** so `docker exec apigenie python -m pytest tests/` works in a freshly-built image without an extra install step. The previous Dockerfile installed runtime deps only.
- **Stale `tests/test_alerts_phase4.py::TestEgressWireContract` assertions aligned** with the deliberate `alerts.build_scope` clamp to `account:site` (introduced in P4.x; the `/v1/alerts` gateway on `usea1-purple` 2026-06-10 was found to silently drop group-scoped sends — see `alerts.build_scope` docstring). The tests still asserted the old `A:S:G` shape; updated to match the new contract.

Breaking changes & migration (v4.0 → v4.1)

The upgrade is **container-only** — no schema migration, no env-var changes, no breaking surface changes. Two operational items to know:

1. **New host port: 4317 / TCP.** OTLP/gRPC ingress. Open it in your firewall / cloud SG if you want collectors to push over gRPC. If you only need OTLP/HTTP, port 443 keeps working with no extra config.
2. **`docker compose up -d --build` is required** (not just `restart`). The new gRPC port mapping on the nginx service and the new `nodejs` + `pytest` packages in the apigenie image need a real image rebuild.

Two **non-breaking but worth knowing** items:

- The Custom Listeners tab still lives in the user portal (`data-portal="user"`) and the wizard reuses the existing `POST /admin/api/listeners` endpoint — so push-sink listeners inherit the same RBAC entitlements (`listeners:create / modify / delete`) and owner/visibility scoping as synthetic and replay listeners. No new permissions.
- `APIGENIE_OTLP_GRPC_ENABLED` (default `true`) opts the gRPC server out entirely — set to `false` if you want to keep port 4317 closed or run in a restricted environment.

Upgrade procedure

```

docker compose down
git pull          # pulls v4.1
docker compose up -d --build

# Confirm both halves of the new listener kind are healthy:
docker exec apigenie python /app/scripts/smoke_otlp.py
# → SMOKE OK    (creates a listener, pushes OTLP/HTTP + OTLP/gRPC, verifies hits, cleans u

# Optional: run the full regression suite – now node + pytest ship in the image
docker exec apigenie python -m pytest tests/ -q
# → all green

# Smoke the egress side end-to-end (creates a listener + an OTLP push profile,
# runs both transports against it, verifies decoded events land, cleans up):
docker exec apigenie python /app/scripts/smoke_otlp_egress.py
# → EGRESS SMOKE OK

```

What's next

- **More OTLP signals on the egress side.** Today push profiles emit only OTLP logs; metrics + traces will follow once the synthetic generators emit non-log shapes (the framework is already signal-aware via `otlp_signal`).
- **OTLP receiver → push profile chaining.** Right now an apigenie deployment can act as a collector *or* a producer; a tiny piece of glue ("forward this push-sink listener's hits to this push profile") would turn it into a full mini-collector with arbitrary transform stages.
- **Per-listener token rotation.** Today the listener's bearer token is a static string; a rolling-rotation policy (next-token + grace window) would close the auth-replay window for long-lived collector deployments.

v4.0 — The Multi-User Edition

Released June 2026. The biggest release since v1.0. ApiGenie is no longer a single-tenant mock server: one deployment now hosts arbitrarily many isolated users, each with their own log profiles, detection rules, source identifiers, SentinelOne console, avatar, and recovery flow — driven from a brand-new `/portal` UI that lives side-by-side with the existing `/admin`.

If you are upgrading from v3.x: read the [Breaking changes & migration](#) section before `docker compose up -d --build`. Most installs need *zero* manual steps, but two env vars and one persistent volume path moved.

At a glance

- **Two portals on the same TLS port.** `/admin` for operators, `/portal` for end-users. Same `ag_session` cookie, two distinct surfaces, role-aware sidebars, server-side enforcement at the HTTP middleware layer.
- **Full RBAC.** Entitlements (named bundles of `{category: [permissions]}`) × five categories × five permissions, with admin implicitly holding every permission. Permission checks are server-side on every write endpoint — not just UI hiding.
- **Per-user identifier matching.** Each pull source stays a single shared endpoint (`/api/v1/logs` , `/web/api/v2.1/threats` , ...). The credential the collector presents (Bearer token, tenant id, API key, ...) is matched against per-user identifiers; the first match wins, and log shaping becomes that user's profile, intensity, detection rules, and entity blending. No match → public profile fallback. Reserved demo tokens are guarded against accidental registration.
- **Per-user SentinelOne console (Phase 3.5).** Every user can set their own S1 URL + API token in **My Account**. Every `/admin/api/s1/*` call from that session — or from an admin acting-as them — automatically routes through *their* tenant. Two SEs sharing one ApiGenie can finally point at their own consoles without coordination.
- **Self-service account settings (Phase 3.5).** `GET /admin/api/me/account` , `PUT /admin/api/me/email` , `PUT /admin/api/me/password` (verifies current), `GET/PUT/DELETE /admin/api/me/s1-console` (token is write-only — never returned).
- **Avatars (Phase 3).** Drop in a PNG / JPEG; Pillow center-crops to a 250×250 circular RGBA portrait. Topbar, switcher, sidebars all light up.
- **Password handoff & recovery (Phase 3).** Admin creates a user without a password, gets a one-shot `/portal/set-password?token=...` link to share over any channel. Same machinery powers admin-issued recovery links. No SMTP required.
- **"Viewing as" switcher.** Admin signed into `/portal` can pick a target user; every owner-scoped read & write happens in that user's namespace. Support-friendly, password-sharing-free.
- **91 regression tests.** Every RBAC guarantee — identifier matching, owner scoping, S1 resolution order, avatar processing, recovery-token lifecycle, self-service semantics, acting-as edge cases — is locked in by `pytest` and ships with the repo.
- **One-page landing redesigned.** The public `/` page now opens a small menu instead of a single Admin button: **User Portal** ("Telemetry config & monitoring") and **Admin** ("Infrastructure & security").
- **New pull source — Mimecast Email Security.** OAuth2 client_credentials → SIEM API 2.0 (`/siem/v1/events/cg`), 8 log types: receipt, process, delivery, AV, spam, TTP URL/Attachment/Impersonation Protect. Plus the 1.0 endpoints aligned to the collector's `ENDPOINT_REGISTRY` .
- **AWS deployment** is now first-class: parametrised Terraform (`terraform/` — EC2 + EIP + SG + IAM + cert-bot bootstrap) and a zero-to-hero `docs/AWS_DEPLOYMENT.md` guide.

What's new in detail

Multi-user & RBAC (Phases 1 – 3.5)

The full design lives in `docs/MULTI_USER_LOG_PROFILING.md` and `docs/RBAC_MODEL.md` . Phased landings:

Phase	What landed	Where it lives
P1 — Identity & RBAC core	SQLite-backed <code>accounts.users</code> , <code>accounts.entitlements</code> , <code>accounts.identifiers</code> , <code>accounts.recovery_tokens</code> . Two-portal auth (<code>/admin/login</code> , <code>/portal/login</code>). Admin user CRUD (<code>/admin/api/rbac/*</code>). Server-side permission model. <code>_portal_role_guard</code> HTTP middleware rejects user-portal sessions reaching admin-only paths. Investigation password gate retired.	<code>accounts.py</code> , <code>app.py</code> , <code>admin.py</code>
P2 — Data ownership	<code>owner_id</code> + <code>visibility</code> (<code>private</code> / <code>public</code>) on Log Profiles, Detection Rules, Log Push Profiles, Custom Listeners, Source↔Profile Bindings. Admin sees everything; users see their own + <code>public</code> . Per-user identifier matching on every pull request (<code>auth.identify_caller</code>). Reserved-credentials guard. User portal substitutes placeholders into Source Details cards so demo tokens never leak.	<code>profiles.py</code> , <code>detection_rules.py</code> , <code>log_pusher.py</code> , <code>listeners.py</code> , <code>auth.py</code> , <code>admin.py</code>
P2.5 — Detection-rule injection scoping	Detection rules respect the resolved caller (owner-or-public). Per-source rules don't bleed across users.	<code>detection_rules.py</code> , <code>tests/test_rbac_phase2_5_detection.py</code>
P3 — Polish	Per-user avatars (Pillow 250x250 circular PNG, max 5 MB input, atomic on-disk store keyed by uid). One-shot admin handoff links + recovery-token lifecycle (<code>POST /admin/api/rbac/users/{uid}/reset-link</code>). Log-Push detection-rule injection is now caller-aware via a <code>ContextVar</code> .	<code>avatars.py</code> , <code>accounts.py</code> , <code>log_pusher.py</code>
P3.5 — Self-service	<code>GET /admin/api/me/account</code> · <code>PUT /admin/api/me/email</code> · <code>PUT /admin/api/me/password</code> (verifies current) · <code>GET/PUT/DELETE /admin/api/me/s1-console</code> . New per-request caller-context middleware so every <code>/admin/api/s1/*</code> call resolves the right console without route-level threading. Acting-as honoured for email/S1; <i>intentionally not</i> honoured for password (so admins can't silently rewrite a target's password through self-service).	<code>accounts.py</code> , <code>s1_detection_library.py</code> , <code>app.py</code> , <code>admin.py</code>

The five RBAC categories (Log Profiles · Detection Rules · Log Push · Custom Listeners · Source Bindings) each support five permissions (View · Create · Modify · Delete · Manage). Six categories remain admin-only and are not in entitlements at all: Intrusions · Investigations & Bans · Container Logs · Observability · System Settings · Entitlement / User management.

New & improved sources

- **Mimecast Email Security** (pull) — OAuth2 `client_credentials` → JWT, SIEM API 2.0 batch fetch at `/siem/v1/events/cg`. 8 log types with realistic per-type weightings: `receipt`, `process`, `delivery`, `av`, `spam`, `ttp-url`, `ttp-attachment`, `ttp-impersonation`. Also exposes the matching 1.0 endpoint shapes the collector's `ENDPOINT_REGISTRY` expects.
- **Microsoft 365** — multi-domain audit logs and tightened source-trace patterns so M365 / Entra ID / Defender for Cloud don't bleed into each other's request inspector groupings.

Hardening, infra & dev-loop

- **AWS deployment.** `terraform/` is now parametrised (EC2 instance type, AMI, EIP, security group, IAM role, optional Route53 record, `bootstrap.sh` runs on first boot to issue Let's Encrypt). `terraform.tfvars.example` lays out every knob. Full walkthrough in [docs/AWS_DEPLOYMENT.md](#).
- **91-test regression suite.** `tests/` ships with the repo and runs inside the container (`docker exec apigenie python -m pytest tests/ -v`). `tests/conftest.py` redirects every storage path to a fresh tmp dir *before* importing project modules, so the suite is hermetic. Coverage map:
 - `tests/test_rbac_phase2.py` — accounts, entitlements, identifiers, reserved-creds, masking, viewing-as.
 - `tests/test_rbac_phase2_5_detection.py` — per-user detection injection (pull).
 - `tests/test_rbac_phase3_log_push.py` — per-user detection injection (Log Push).
 - `tests/test_rbac_phase3_avatars.py` — Pillow pipeline, store, endpoints.
 - `tests/test_rbac_phase3_recovery.py` — recovery-token lifecycle.
 - `tests/test_rbac_phase35_self_service.py` — email / password / per-user S1 helpers.
 - `tests/test_rbac_phase35_endpoints.py` — `/admin/api/me/*` + caller-context middleware.
- **Dashboard JS parse-check.** `scripts/check_dashboard_js.py` boots `admin.py` against a tmp data root, renders the dashboard HTML, extracts the inline JS and parses it with `acorn` (when available) or `node --check` — turns silent template-literal bugs into a hard failure during CI.
- **Dockerfile.** Adds Pillow (for avatars) and `pytest` + `pytest-asyncio` (so the test suite runs against the actual container).

Documentation overhaul

The README, both lab guides, and the design doc were rewritten end-to-end for the multi-user world. New top-level resources:

- [README.md](#) — new **Multi-user & RBAC** section with the RBAC quickstart, refactored Portals section covering both UIs, completely rewritten endpoint catalogue (Identity & RBAC / Telemetry config / S1 / Diagnostics), per-row admin-only flag, refreshed env-var + storage tables.
- [docs/USER_GUIDE.md](#) — Section 0 (single-user lab) + 11 RBAC exercises ending with **Exercise 11 — Account Settings** (Phase 3.5).

- [docs/ADMIN_GUIDE.md](#) — Section 0 (admin RBAC lab) + Exercises A – I, with **Exercise I — Audit per-user S1 console overrides**. The §11 API reference documents the source-data vs control-plane split, the `/admin/api/me/*` family with acting-as semantics, and the global vs per-user S1 resolution order.
- [docs/MULTI_USER_LOG_PROFILING.md](#) — full Phase 1 – 3.5 design rationale.
- [docs/RBAC_MODEL.md](#) — categories × permissions reference card.
- [docs/RBAC_USER_PROFILES.md](#) — owner-scoping rules per resource family.
- [docs/AWS_DEPLOYMENT.md](#) — Terraform walkthrough.

Breaking changes & migration (v3.0 → v4.0)

The upgrade story is "down, pull, up": container-level state lives in one Docker volume and the schema bootstraps itself on first start. But three things shifted in ways that may surprise long-running deployments:

1. **Investigation-password gate removed.** The 🔍 Investigations tab was previously protected by a second password (`APIGENIE_INVESTIGATE_PASSWORD` , `./data/investigate_pass`). It is now plain admin-only. The env var and file are still tolerated — you can delete them at your leisure — but they no longer control access. If your runbooks reference that password, simplify them.
2. **Two new SQLite files in the data volume.**
 - `./data/apigenie.db` — accounts, entitlements, identifiers, recovery tokens, per-user S1 settings.
 - `./data/avatars/<uid>.png` — one PNG per user that has uploaded an avatar. Make sure your `./data` host directory is a **named Docker volume** (it is by default in the shipped `docker-compose.yaml`). A bind mount that doesn't persist across `docker compose down` will lose every user — same trap as `./data/profiles/` in v3.
3. **Per-user log shaping requires identifier registration.** In v3.x, every collector that hit `/api/v1/logs?...` saw the public profile binding. In v4.0, that still happens for the reserved demo tokens (`apigenie-valid-token-001` ...) and any unknown credential — but a user that wants their *own* shaping has to register the credential value under **User Portal** → **Source Identifiers** (or `POST /admin/api/identifiers`). Existing demos using only the reserved tokens are unaffected.

There are also four **non-breaking but worth knowing** items:

- The Admin UI sidebar grew a **My Account** entry (Phase 3.5). The built-in admin sees a stub there — admin password changes still happen in **System Settings** → **Change admin password** because the built-in admin has no DB row.
- `ADMIN_PASSWORD_FILE` (default `/var/lib/apigenie/admin_pass`) is now consulted ahead of `ADMIN_PASSWORD_HASH` . Behaviour matches v3 — the file is only written when an admin actively changes the password through the UI — but the env var is now documented as a debugging knob, not the primary path.
- The data-root env-var triplet (`APIGENIE_DATA_ROOT` / `APIGENIE_DATA_DIR` / `APIGENIE_DATA`) is unchanged from v3, but the README's env table now spells out which module reads which. Tests rely on this; production usually doesn't have to touch it.
- The public landing page (/) shows a **dropdown** instead of a single "Admin" button. Bookmarks to `/admin/login` and `/portal/login` continue to work.

Upgrade procedure


```

docker compose down
git pull          # pulls v4.0
docker compose up -d --build
# That's it. The accounts DB is created automatically the first time
# the container boots; the admin password remains whatever it was.

# Optional: confirm everything is green
docker exec apigenie pip install --quiet pytest pytest-asyncio
docker exec apigenie python -m pytest tests/ -v
# → 91 passed

```

If anything looks wrong post-upgrade, the **first** diagnostic is `GET /admin/api/me` while signed into both portals — it returns the resolved identity, role, effective permissions, and `has_avatar` flag, and is the cheapest possible "is the session what I think it is?" check.

What's next

- **Hyperautomation / SOAR integration.** Letting workflows mutate ApiGenie state (publish a profile, mint a recovery link, rotate a per-user S1 token) over a single signed webhook.
- **Bulk user import** (CSV / SCIM-lite) for SE workshops with 50+ attendees.
- **Per-user log push destinations** so each user can pipe their own generated logs into their own collector without exposing global push profiles.

If you have an opinion on any of those — or a use case the current model doesn't cover — open an issue and tag it `v4.x`.

v3.0 — *SentinelOne edition* — April 2026

- **SentinelOne pull source** (`/web/api/v2.1/threats` , `/activities` , `/agents`) — full MITRE ATT&CK mapping, cursor pagination, real console response shape.
- **6 new pull sources:** Cato Networks SASE, Cloudflare, Zscaler ZPA, Corelight/Zeek, CyberArk EPM/PAM, Stamus Networks SSP.
- **SentinelOne Singularity push source + Attack Scenario Builder** (multi-event correlated attacks, ATT&CK-anchored).
- **Cisco / HPE Aruba switch push sources** (port security, STP, 802.1X, RADIUS, VSF, PoE).
- **S1 Detection Library integration** (`/admin/api/s1/*`) — query catalog + custom rules, preview-import to local detection rules, enable/disable on S1 from ApiGenie. (*In v4.0 this becomes per-user-aware.*)
- **HEC transport fix** — `http.client` rewrite, scheme stripping, auth-header auto-detection.

v2.x — *Detection & Push* — February – March 2026

- **Detection Rules** — count-based (1 in N) and time-based (every N s) injection of SIEM-triggering log patterns into the normal event flow. Works across all 14 HTTP sources + Kafka + Pub/Sub

publishers.

- **Log Push framework** — 10 vendor generators (Palo Alto, FortiGate, Check Point, Cisco ASA, CrowdStrike, Carbon Black, Zscaler, Imperva, Barracuda, Infoblox), 3 formats (JSON / Syslog RFC5424 / CEF), 3 transports (HTTP / Splunk HEC / Syslog TCP-or-UDP). Start/stop control, last-100 event log with delivery confirmation, per-profile TLS.
- **Microsoft 365 pull source** — 14 event categories (Mailbox, ATP, DLP, eDiscovery, Admin, SPO/OneDrive, Teams, OAuth, Inbox rules, Power Platform, PIM, Audit search, Quarantine, Login). Two modes: Graph `alerts_v2` and Management Activity API.
- **Per-source log volume control** (1 – 100 %) so different consumers can be load-balanced on the same deployment.
- **Architecture hub-and-spoke diagram** on the public landing.
- **SASL/OAUTHBEARER** support for the Azure Event Hubs Kafka listener.

v1.x — Foundations

- Self-contained mock server for 14 security platform APIs (Okta, Netskope, Entra ID, Defender for Cloud, Cisco Duo, Tenable, Proofpoint, Wiz, Snyk, Darktrace, plus AWS generators).
 - Azure Event Hubs (Kafka SASL/SSL + SASL/PLAINTEXT) and GCP Cloud Logging (Pub/Sub emulator over TLS) streaming sources with background publishers.
 - One-shot `./scripts/bootstrap.sh` : domain, admin password, TLS mode (self-signed / Let's Encrypt / existing), all in a single Docker Compose stack.
 - Admin UI with Request Inspector, Observability (Flows / GeoMap / Usage / System), Intrusions, Investigations, Container Logs, Listeners, Log Profiles, Source Details, System Settings.
 - Persistent telemetry (`telemetry.db` , 1-year retention, adaptive bucketing 1 min → 1 day).
 - In-stack certbot sidecar with zero-downtime nginx reload + Kafka restart on renewal.
-

Versions before v3.0 were tagged informally inside the SE team; v4.0 is the first release with a published release-notes document. Future versions will append above this line.